CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Experiment 08

Name	: Punit Kumar	**UID**	: 23BCS14148
Branch	: CSE	**Section**	: KRG-2A
Semester	: Sixth	**Date of exp.**	:08/04/2026
Subject	: System Design	**Subject Code**	: 23CSH-314

AIM

To design a stock-trading platform (Similar to Zerodha, Grow & Upstock)
These platforms serves as the commission-free online platform for trading and monitoring of the stocks.

OBJECTIVE:

- Enable secure user onboarding and authentication with KYC/AML compliance integrated into the system design.
- Ingest and process real-time market data feeds from exchanges with scalable, fault-tolerant streaming architecture.
- Design a trading engine microservice to handle order placement, modification, cancellation, and exchange integration.
- Implement portfolio and settlement services to track holdings, P&L, and clearing house interactions.
- Ensure scalability, reliability, and compliance through event-driven architecture, audit logging, and high-availability deployment.

PROCEDURE

- Studied real-world trading to understand their architecture and workflows.
- Identified core entities like User, Account, Order, Trade, Portfolio, Market Data etc.
- Analyzed real-time market data ingestion using streaming systems (Kafka, WebSockets) for quotes and order books.
- Collected functional and non-functional requirements including compliance, latency, scalability, and fault tolerance.
- Designed APIs for :place order, cancel order, portfolio view, market data query etc..
- Evaluated order matching and risk-management algorithms for margin checks, exposure limits, and compliance.
- Analyzed consistency and reliability constraints in distributed trade execution and settlement systems.

Core-Entities of the System:

-Users	-Trade
-Portfolio	-Watchlist
-Stocks	-Orders

FUNCTIONAL REQUIREMENTS

a) Users should be able to do the registration, login and KYC verification.
b) Application should allow users to view real time stocks prices & its historical data.
c) Application should support placing, modifying, and cancelling orders along with limiting orders with accurate status updates.
d) Application should provide users with a watchlist with real time updates.
e) Application should provie a dashboard to track current holdings, trade history, P& L, and performance insights.

NON-FUNCTIONAL REQUIREMENTS

a) Scale: 2 - 3M DAU with high-frequency trades, along with 8 - 10k stocks.
b) CAP Theorem: Consistency >>> Availability
    i. Correctness of stocks is more important than uptime. (A wrong balance, duplicate trade, or stale price can lead to huge financial losses)
    ii. Getting the stocks prices in real-time (highliy available)

c) Latency: Order placement < 100ms, market data (updated stock value) < 50ms

API DESIGN

Users:

1. POST: / auth / signup
2. POST: / auth / verify-KYC
3. POST: / auth / login

Market Data:

1. [Listing of all the stocks] WS: / api / v1 / stocks
2. [stock details] GET: / api / v1 / stocks / (stock_symbol_id)
3. [Historical Data] GET: / api/ v1 / stocks / {stock_symbol_id} / history

Funds:

1. POST: / api / v1 / funds / deposit
2. POST: / api / v1 / funds / withdraw
3. GET: / api / v1 / funds / history

Trading:

1. [Place buy / sell order] POST: / api / v1 / orders
2. [List user orders] GET: / api / v1 / {user_id} / orders
3. [Order Status] GET: / api / v1 / orders / {order_id}
4. [Cancel order] DELETE: / api / v1 / orders / [order_id]

Portfolio:

1. GET: / api / v1 / portfolio
2. GET: / api / v1 / portfolio / holdings
3. GET: / api / v1 / portfolio / positions
4. GET: / api . v1 / portfolio / performance [OPTIONAL]

Watchlist:

1. GET: / api / v1 / watchlists
2. POST / api / v1 / watchlists / {watchlist_id} / symbols

HIGH-LEVEL DESIGN

For reference please view the attached stock-trade.io file on draw.io(diagrams.net).

LEARNING OUTCOMES

- Understood the architecture of real-world trading systems and how they integrate with exchanges and clearing houses.
- Identified and modeled core entities such as User, Account, Order, Trade, Portfolio, Market Data, and Settlement.
- Learned how to design scalable APIs for trading operations, portfolio management, and real-time market data queries.
- Explored risk management and compliance mechanisms including margin checks, exposure limits, and audit logging.
- Developed insights into distributed system constraints like consistency, fault tolerance, and high availability in financial platforms.